



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

COMPARITIVE STUDY OF GROWTH FUNCTIONS

CSA0607- Design and Analysis Of Algorithms

PRESENTED BY :

A.Sai Joshitha (192424220)

GUIDED BY :

Dr.H.Riasudheen

INTRODUCTION

What are Growth Functions?

- Growth functions describe how an algorithm's running time increases as input size increases.
- They help compare algorithm efficiency.
- Independent of hardware and programming language.
- Lower growth rate = Better scalability.

Why Compare Growth Functions?

Importance:

- Predicts algorithm performance.
- Helps choose efficient algorithms.
- Measures scalability.
- Reduces execution time.
- Optimizes resource usage.

Common Growth Functions

Growth Function	Complexity
Constant	$O(1)$
Logarithmic	$O(\log n)$
Linear	$O(n)$
Linearithmic	$O(n \log n)$
Quadratic	$O(n^2)$
Cubic	$O(n^3)$
Exponential	$O(2^n)$
Factorial	$O(n!)$

Growth Order

Best → Worst

$O(1)$



$O(\log n)$



$O(n)$



$O(n \log n)$



$O(n^2)$



$O(2^n)$

Comparison Table

Complexity	Example
$O(1)$	Array Access
$O(\log n)$	Binary Search
$O(n)$	Linear Search
$O(n \log n)$	Merge Sort
$O(n^2)$	Bubble Sort
$O(2^n)$	Recursive Fibonacci

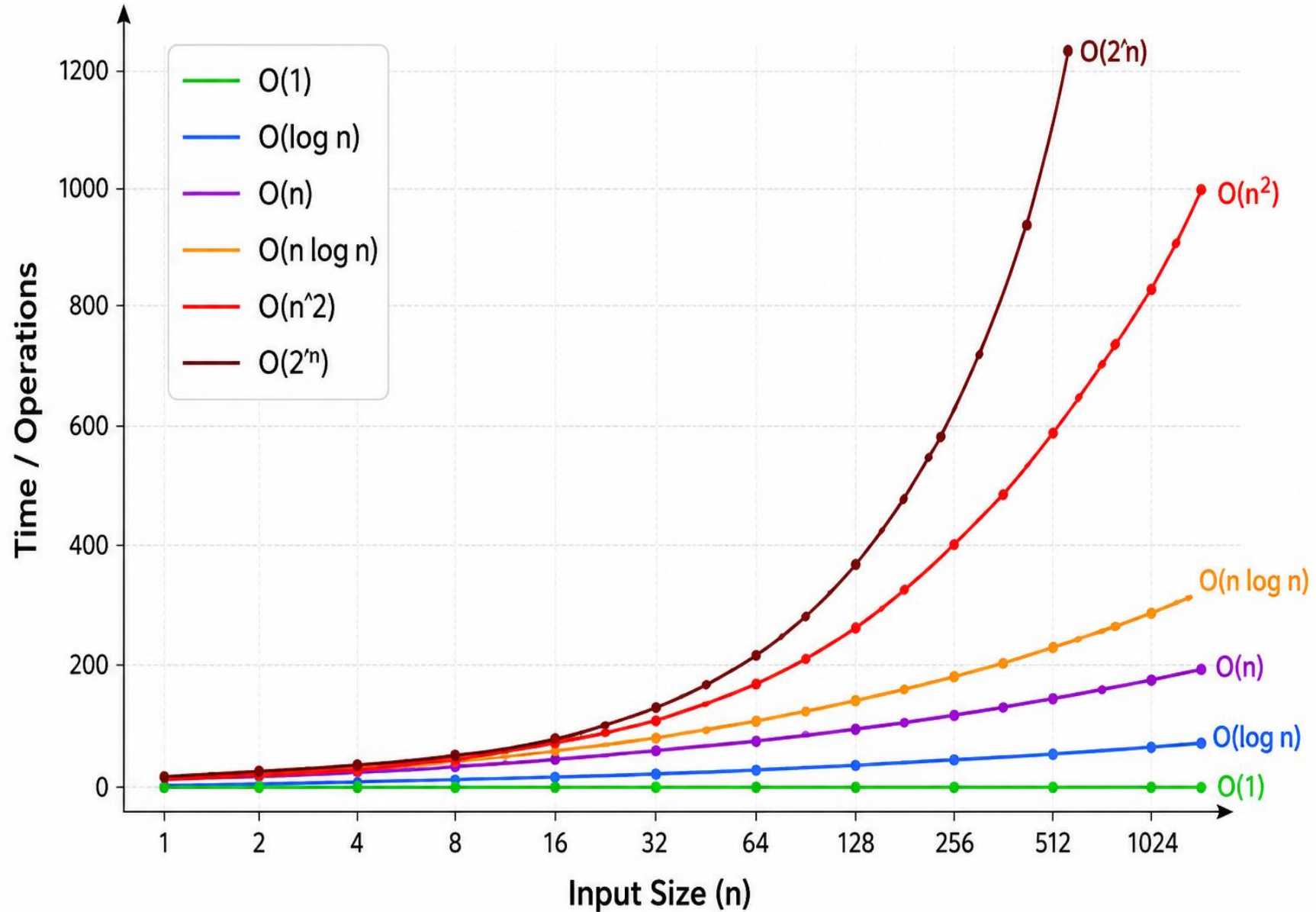
Advantages of Low Growth Functions

- Faster execution
- Better scalability
- Lower memory usage
- Suitable for large datasets
- Improved user experience

Real-Life Examples

- 💡 **$O(1)$ – Constant:** Turning a light switch on or off.
- 📖 **$O(\log n)$ – Logarithmic:** Finding a word in a dictionary.
- 👤 **$O(n)$ – Linear:** Searching for a student's name in an attendance list.
- 📊 **$O(n \log n)$ – Linearithmic:** Sorting exam marks from lowest to highest.
- 🌸 **$O(n^2)$ – Quadratic:** Comparing every student with every other student in a class.

COMPARISON OF GROWTH FUNCTIONS



CONCLUSION

- Growth functions measure algorithm efficiency.
- Lower-order complexities perform better on large inputs.
- $O(1)$, $O(\log n)$, and $O(n \log n)$ are preferred.
- $O(2^n)$ and $O(n!)$ should be avoided for large datasets.

REFERENCES

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms (4th Edition), MIT Press.
- Weiss, M. A. Data Structures and Algorithm Analysis, Pearson.
- GeeksforGeeks – Data Structures and Algorithms.
- TutorialsPoint – Data Structures & Algorithms Tutorials.
- Programiz – Algorithms and Time Complexity Tutorials.

THANK YOU